Efficiently Deleting Matches from a Linked List

Kevin Browne

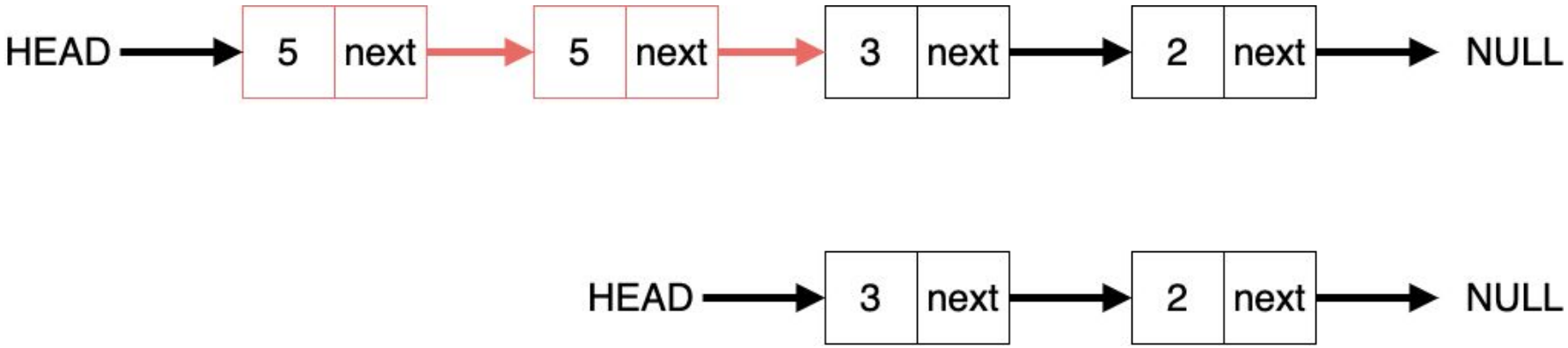
Efficiently deleting matches

- How can we delete matching nodes from a list while traversing it only once?
- When we encounter a matching node, we should only advance when subsequent nodes no longer match
- i.e. we should effectively pause our traversal **until** the next node does not match, otherwise so long as the next node matches we should delete it

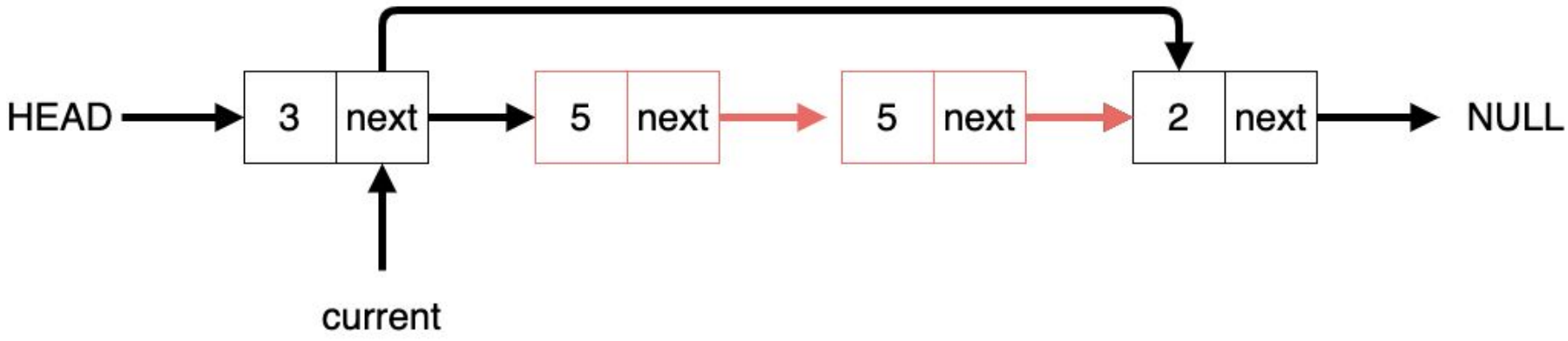
Efficiently deleting matches

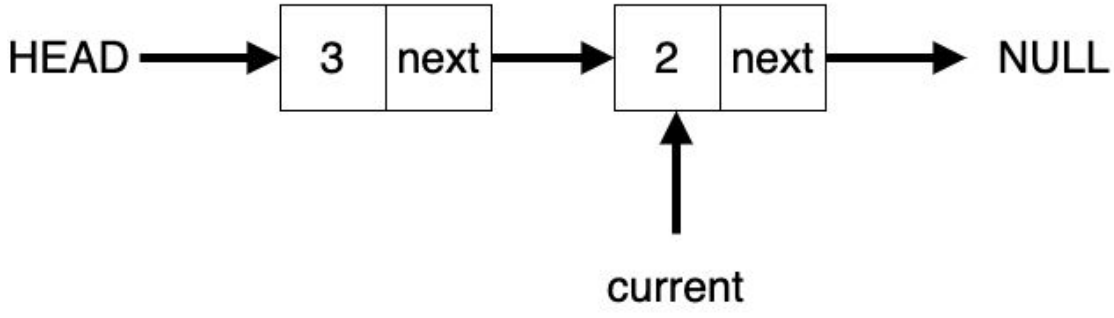
- In the case of head node, this means we should continuously delete the head node until the current head node no longer matches
  - And this node will become the ***new*** head of the list
- In the case of any remaining nodes, this means we should continuously delete the next node until the next node no longer matches
  - And at this point, we can traverse our pointer to the next node, and start looking for matches again

Deleting multiple matching nodes at the head of the list...



Deleting multiple matching nodes fro the middle of the list...





Continuously deleting matching head nodes...

```
Node* efficient_delete_match(Node *head, int delete_value, int *num_deleted)
{
  if (head == NULL) return NULL ;

  Node *current, *temp, *new_head;
  current = head;
  *num_deleted = 0;

  while (current->value == delete_value)
  {
    temp = current;
    current = current->next;
    free(temp);
    *num_deleted = *num_deleted + 1;

    if (current == NULL) return NULL;
  }
  new_head = current;
```

Continuously deleting the next node if it matches before traversing the list

```
    while (current->next != NULL)
    {
        if (current->next->value == delete_value)
        {
            temp = current->next;
            current->next = current->next->next;
            free(temp);
            *num_deleted = *num_deleted + 1;
        }
        else current = current->next;
    }

    return new_head;
}
```

Testing the function…

```
printf("\n\nefficient_delete_match test\n");
printf("****************************************\n");
Node *list8 = NULL;
list8 = insert_at_head(list8, 4);
list8 = insert_at_head(list8, 3);
list8 = insert_at_head(list8, 4);
list8 = insert_at_head(list8, 5);
list8 = insert_at_head(list8, 4);
list8 = insert_at_head(list8, 4);
list8 = insert_at_head(list8, 7);
list8 = insert_at_head(list8, 4);
list8 = insert_at_head(list8, 4);
printf("\nList 8...\n");
print_list(list8);
int num_deleted = 0;
list8 = efficient_delete_match(list8, 4, &num_deleted);
printf("\nList 8 after delete...\n");
print_list(list8);
printf("Number of elements deleted: %d\n\n\n", num_deleted);
```

Test results

```
efficient_delete_match test
*************************************

List 8...
Node 0: 4
Node 1: 4
Node 2: 7
Node 3: 4
Node 4: 4
Node 5: 5
Node 6: 4
Node 7: 3
Node 8: 4

List 8 after delete...
Node 0: 7
Node 1: 5
Node 2: 3
Number of elements deleted: 6
```

Performance test comparison

- Let’s do a performance test comparison between our old and new delete all matches functions!
- We’ll create massive linked lists (50k nodes) with many nodes to be delete
- We’ll time their performance using clock() values
  - We’ll need to include time.h to use clock()

Performance test code…

```
printf("Performance test of delete functions\n");
printf("*****************************************\n\n");
Node *list9 = NULL, *list10 = NULL;
for (int i = 0; i < 50000; i++) list9 = insert_at_head(list9, i % 10);
for (int i = 0; i < 50000; i++) list10 = insert_at_head(list10, i % 10);
clock_t tic, toc;
tic = clock();
list9 = delete_all_matches(list9, 4, &num_deleted);
toc = clock();
printf("delete_all_matches: %fs\n", (double)(toc - tic) / CLOCKS_PER_SEC);
printf("elements deleted: %d\n\n", num_deleted);
tic = clock();
list10 = efficient_delete_match(list10, 4, &num_deleted);
toc = clock();
printf("efficient_delete_match: %fs\n", (double)(toc - tic) / CLOCKS_PER_SEC);
printf("elements deleted: %d\n\n", num_deleted);
```

Performance test results…

```
Performance test of delete functions
*************************************

delete_all_matches: 0.333229s
elements deleted: 5000

efficient_delete_match: 0.000477s
elements deleted: 5000
```